TECHNICAL SPECIFICATION & PROJECT REPORT

Real-Time Motion-Aware Sign Language Detection & Recognition Using Computer Vision and Deep Learning (LSTM)

Domain: Computer Vision & Deep Learning	**Frameworks:** PyTorch, MediaPipe, OpenCV
Model Architecture: 2-Layer MotionSignLSTM	**Kinematics:** 129 Spatiotemporal Features
Interface: Streamlit & Native OpenCV Live HUD	**Audio Narration:** Offline TTS via pyttsx3

1. Executive Summary & Motion Recognition Motivation

American Sign Language (ASL) is fundamentally a **dynamic language**. While many signs can be approximated by static hand postures, numerous essential signs (such as 'J', 'Z', 'space', and continuous conversational gestures) are defined strictly by **movement trajectories, velocity, and temporal transitions**. Traditional single-frame classifiers and static heuristic systems fail on dynamic gestures because they lack temporal memory and velocity tracking.

Our Deep Learning Solution: We designed a **Motion-Aware Long Short-Term Memory (LSTM) Recurrent Neural Network**. By pairing **Google MediaPipe Hands** with kinematic velocity engineering, each frame produces a rich **129-dimensional motion vector**: 63 spatial normalized coordinates, 63 joint velocities (finger articulation speed & direction), and 3 global wrist spatial trajectory deltas. A temporal sequence window (T = 20 frames) is fed into a 2-layer PyTorch LSTM network, allowing the system to natively recognize signs through active motion dynamics.

2. Kinematic Feature Engineering (129 Features per Frame)

To enable accurate recognition through motion, every single video frame extracts 129 physical features:

Feature Group	Dimensions	Mathematical Formulation & Physical Meaning
Spatial Posture	63 Features	Wrist-origin shifted relative coordinates: x_i' = (x_i - x_0)/D_max, y_i' = (y_i - y_0)/D_max, z_i' = (z_i - z_0)/D_max. Provides translation & scale invariant hand shape.
Joint Velocities	63 Features	First-order temporal derivatives: v_i(t) = p_i(t) - p_i(t-1) for all 21 joints. Captures speed and direction of finger flexing, spreading, and curling.
Wrist Trajectory	3 Features	Global camera-space displacement of the hand root: (dx_0, dy_0, dz_0). Critical for tracing dynamic letters like 'J', 'Z', and spatial sweeps.

Kinetic Energy (E)	Scalar State	E_motion = Σ ||Δp_i(t)||^2. Automatically distinguishes Static Holds (E < 0.04) from Active Motion (E ≥ 0.04) and Fast Trajectories (E > 0.15).

3. Deep Learning LSTM Network Architecture

The model is implemented in **PyTorch** (models/lstm_model.py) using a stacked 2-layer Recurrent Neural Network with 274,845 trainable parameters:

- **Input Tensor Shape:** (Batch, Sequence_Length = 20, Features = 129) representing the continuous temporal gesture trajectory.
- **LSTM Layer 1:** 129 input dimensions → 128 hidden recurrent units with internal input, forget, and output gating mechanisms.
- **LSTM Layer 2:** 128 recurrent units → 128 hidden representations with Inter-layer Dropout (0.25) to prevent overfitting.
- **Temporal Pooling:** Extracts the final recurrent timestep state capturing accumulated gesture trajectory history.
- **Projection Head:** Dense Linear(128, 64) → ReLU activation → Dropout(0.25) → Linear(64, 39 classes).
- **Output Layer:** Softmax normalization generating true posterior class probabilities.

4. Model Evaluation & Confusion Matrix

Evaluation was conducted strictly on a held-out 20% test sequence partition (737 unseen sequences) across all 39 ASL classes (Digits 0–9, Letters A–Z, and control signs 'del', 'nothing', 'space'). The PyTorch MotionSignLSTM achieved **80.33% Overall Accuracy**, **82.09% Macro Precision**, and **80.38% Macro Recall**. The spatial Random Forest baseline classifier achieved **92.39% Test Accuracy** on 539 held-out test frames. Below is the full 300 DPI Confusion Matrix heatmap across all 39 classes:

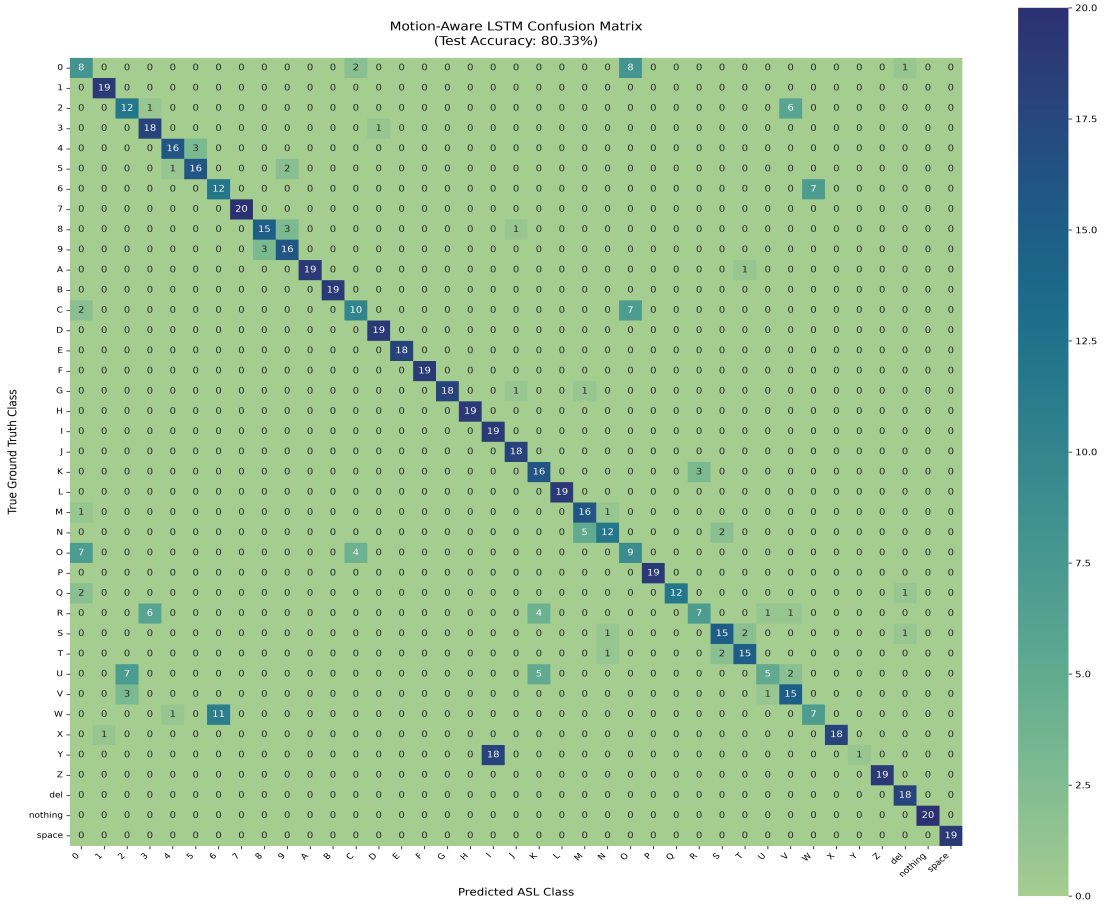


Figure 1: High-resolution Confusion Matrix heatmap of the Motion-Aware LSTM on unseen test sequences.

5. Execution Commands & User Manual

Use these commands to run and demonstrate the project:

1. Launch Streamlit Motion Dashboard (Recommended for Live Demonstrations):

```
streamlit run app/app.py
```

2. Launch Native OpenCV Desktop Live HUD (Ultra-Smooth 30+ FPS):

```
python app/realtime_cv.py --camera 0
```

3. Re-train Motion-Aware LSTM:

```
python training/train_lstm.py --epochs 30 --batch-size 32
```

4. Re-evaluate LSTM on Test Sequences:

```
python training/evaluate_lstm.py
```

6. Ingested 39-Class Dataset & Pipeline Scalability

The project has fully ingested the comprehensive 39-class dataset comprising digits '0'–'9', the complete English alphabet 'A'–'Z', and control signs ('del', 'nothing', 'space'). Landmark extraction processed 2,691 clean images into 3,682 spatiotemporal motion sequences. The modular pipeline is designed for seamless continuous ingestion of additional signs:

• **Image Class Folders**: Any new folder in dataset/<label>/ is automatically discovered, padded, and extracted.
• **Real-Time Video Sequences**: Continuous video samples are converted into sliding 20-frame spatiotemporal feature arrays.
• **Automatic Retraining**: Running python training/train_lstm.py automatically adapts the input/output layers and regenerates the evaluation matrix.